

Objetos: iterar propiedades, desestructurar, borrar propiedades, copia superficial y profunda

Object.entries: Para recorrer las claves y valores de un objeto se puede hacer así:

```
let obj={nombre:"Juan",edad:40,profesión:"fontanero"}
for (let [clave,valor] of Object.entries(obj)){
  document.getElementById("demo").innerHTML+=
    "La propiedad " + clave + " vale " + valor; //valor es igual que obj[key]
}
// saldría La propiedad nombre vale Juan, La propiedad edad vale 40...
```

Desestructurar o deconstruir un objeto:

```
let {nombre,edad,profesión}=obj
console.log(nombre) //sale "Juan" (sin poner obj delante)
console.log(edad) //sale 40
console.log(profesion) //sale "fontanero"
```

delete: Borrar una propiedad.

```
delete obj.profesion // obj quedaría {nombre:"Juan",edad:40}
```

Igualar un objeto (copia superficial): cuando hacemos lo siguiente: `let obj2=obj`

`obj` no guarda directamente su contenido sino la dirección de memoria en la que está almacenado su contenido, por lo tanto `obj2` apunta al mismo contenido que `obj`, por tanto son en realidad el mismo objeto, no son 2 copias, 2 objetos, sino un solo contenido.

`obj2.nombre="luis"` hace que `obj.nombre` sea "luis" también

Copia profunda de un objeto : funciona igual que la copia profunda de arrays:

```
let obj2=structuredClone(obj)
let obj2=PARSE.json(PARSE.stringify(obj))
```


Objetos: hacer objetos inmutables

Hacer un objeto inmutable, es hacer que no se puedan crear propiedades o borrar o modificar propiedades existentes, para ello se utilizan estos métodos:

Object.freeze(): impide crear propiedades nuevas, modificar y borrar las existentes

```
let obj={nombre:"Juan",edad:40,profesión:"fontanero"}
Object.freeze(obj)
obj.nombre="Luis" // no modifica la prop nombre, aunque no da error.
obj.provincia="Madrid" // no crea la prop provincia, aunque no da error
delete obj.edad //no borra la propiedad edad, aunque no da error.
//obj vale {nombre:"Juan",edad:40,profesión:"fontanero"}
```

Object.seal(): no permite crear propiedades nuevas, ni borrar propiedades existentes, pero sí permite modificar los valores de las propiedades existentes.

```
let obj2={nombre:"Juan",edad:40,profesión:"fontanero"}
Object.seal(obj2)
obj2.nombre="Luis" // modifica la prop nombre
obj2.provincia="Madrid" // no crea la prop provincia, aunque no da error
delete obj2.edad //no borra la propiedad edad, aunque no da error.
//obj2 vale {nombre:"Luis",edad:40,profesión:"fontanero"}
```

Object.preventExtensions(): no permite crear propiedades nuevas, pero sí borrar propiedades existentes, y también modificar propiedades.

```
let obj3={nombre:"Juan",edad:40,profesión:"fontanero"}
Object.preventExtensions(obj3)
obj3.nombre="Luis" // modifica la prop nombre
obj3.provincia="Madrid" // no crea la prop provincia, aunque no da error
delete obj3.edad //borra la propiedad edad
//obj3 vale {nombre:"Luis",profesión:"fontanero"}
```


FOLDERS

- ▼ Videoconferencias
 - ▶ Array bidimensional
 - ▶ Metodos array y objeto JSON vueltos
 - ▶ Métodos de array map- filter-reduce-every-some-find-copiar
 - ▶ Objetos creados por el usuario y objetos JSON
 - ▼ Objetos-iterar y borrar prop-desestructurar-copiar-inmutables
 - ▶ borrar
 - ▼ Ejercicios
 - <> Ejercicio - copia objeto.html
 - <> Ejercicio - ejemplos iterar-desestructurar-borrar-copiar.html
- 101
011 Presentacion UT4.pdf

```

7 <body>
8 <div id="demo"></div>
9 <script>
10   let obj={nombre:"Juan",edad:40,profesion:"fontanero"}
11   demo.innerHTML+="Recorriendo las propiedades ...<br>"
12   for (let [clave,valor] of Object.entries(obj)){
13     demo.innerHTML+="La propiedad "+clave+" vale "+valor+"<br>"
14   }
15   demo.innerHTML+="<br>Recorriendo las propiedades de otra forma ...<br>"
16   for (let campo of Object.entries(obj)){
17     demo.innerHTML+="La propiedad "+campo[0]+" vale "+campo[1]+"<br>"
18   }
19
20   demo.innerHTML+="<br>Deestructurando el objeto ...<br>"
21
22   let {nombre,edad,profesion}=obj;
23   demo.innerHTML+="nombre vale "+nombre+"<br>"
24   demo.innerHTML+="edad vale "+edad+"<br>"
25   demo.innerHTML+="profesion vale "+profesion+"<br>"
26
27   demo.innerHTML+="<br>Borrando una propiedad del objeto ...<br>"
28
29   delete obj.profesion
30
31   demo.innerHTML+="obj vale: "+JSON.stringify(obj)+"<br>"
32
33   demo.innerHTML+="<br>Copia superficial ...<br>"
34
35   let obj2=obj;
36   obj2.provincia="Almeria"
37
38   demo.innerHTML+="obj vale"

```


FOLDERS

- ▼ Videoconferencias
 - ▶ Array bidimensional
 - ▶ Metodos array y objeto JSON vuelos
 - ▶ Métodos de array map- filter-reduce-every-some-find-copiar
 - ▶ Objetos creados por el usuario y objetos JSON
 - ▼ Objetos-iterar y borrar prop-desestructurar-copiar-inmutables
 - ▶ borrar
 - ▼ Ejercicios
 - <> Ejercicio - copia objeto.html
 - <> Ejercicio - ejemplos iterar-desestructurar-borrar-copiar.html
- 101 Presentacion UT4.pdf

```
Ejercicio - copia array.html | Ejercicio - copia objeto.html | Ejercicio - ejemplos iterar-desestructurar-borrar-copiar.html | en arrays.html
26
27 demo.innerHTML+="<br>Borrando una propiedad del objeto ...<br>"
28
29 delete obj.profesion
30
31 demo.innerHTML+="obj vale: "+JSON.stringify(obj)+"<br>"
32
33 /* demo.innerHTML+="<br>Copia superficial ...<br>"
34
35 let obj2=obj;
36 obj2.provincia="Almeria"
37
38 demo.innerHTML+="obj vale: "+JSON.stringify(obj)+"<br>"
39 demo.innerHTML+="obj2 vale: "+JSON.stringify(obj2)+"<br>"
40 */
41 demo.innerHTML+="<br>Copia profunda utilizando structuredClone...<br>"
42
43 let obj2=structuredClone(obj);
44 obj2.provincia="Almeria"
45
46 demo.innerHTML+="obj vale: "+JSON.stringify(obj)+"<br>"
47 demo.innerHTML+="obj2 vale: "+JSON.stringify(obj2)+"<br>"
48
49 demo.innerHTML+="<br>Copia profunda utilizando JSON.parse(JSON.stringify(
50 obj))...<br>"
51
52 let obj3=JSON.parse(JSON.stringify(obj))
53 obj3.provincia="Almeria"
54
55 demo.innerHTML+="obj vale: "+JSON.stringify(obj)+"<br>"
56 demo.innerHTML+="obj3 vale: "+JSON.stringify(obj3)+"<br>"
```


Ejercicio: copia de objetos

1. Crea un objeto con los siguientes campos: nombre y provincia.
2. Crea una función que ha de crear otro objeto a partir de éste, pero añadiendo el campo país con valor “España”. Después ha de mostrar por consola el objeto original y la copia. Esta función ha de crear el objeto nuevo copiando el original con `copia=original`
3. Crea una función que ha de crear otro objeto a partir de éste, pero añadiendo el campo país con valor “España”. Después ha de mostrar por consola el objeto original y la copia. Esta función ha de crear el objeto nuevo copiando el original con `copia=structuredClone(original)`
4. Crea una función que ha de crear otro objeto a partir de éste, pero añadiendo el campo país con valor “España”. Después ha de mostrar por consola el objeto original y la copia. Esta función ha de crear el objeto nuevo copiando el original con `copia=JSON.parse(JSON.stringify(original))`

FOLDERS

- ▼ Videoconferencias
 - ▶ Array bidimensional
 - ▶ Metodos array y objeto JSON vuelos
 - ▶ Métodos de array map- filter-reduce-every-some-find-copiar
 - ▶ Objetos creados por el usuario y objetos JSON
 - ▼ Objetos-iterar y borrar prop-desestructurar-copiar-inmutables
 - ▶ borrar
 - ▼ Ejercicios
 - <> Ejercicio - copia objeto.html
 - <> Ejercicio - ejemplos iterar-desestructurar-borrar-copiar.html
- 101 Presentacion UT4.pdf

```
1 <!DOCTYPE html>
2 <html>
3 <head>
4 <meta charset="utf-8" />
5 <title>Ejercicio copia objetos</title>
6 </head>
7 <body>
8   <input type="button" value="Crear objeto con copia=obj" onclick="funcionObj1(
9     persona)"><br>
10 <div id="demo"></div>
11 <script>
12   let persona={nombre:"Ana",provincia:"Almeria"}
13   function funcionObj1(obj){
14     let copia=obj;
15     copia.pais="España"
16     console.log("Original")
17     console.log(persona)
18     console.log("Copia")
19     console.log(copia)
20   }
21 </script>
22
23 </body>
24 </html>
25
```




F:\Centro IES Virgen de la Paz\Materiales Maite\DAW\DWECC distancia\UT04\Videoconferencias\Objetos-iterar y borrar prop-desestructurar-copiar-inmutables\Ejercicios\Ejercicio - copia objeto.html • (Videoconfe...

File Edit Selection Find View Goto Tools Project Preferences Help

FOLDERS

- ▼ Videoconferencias
 - ▶ Array bidimensional
 - ▶ Metodos array y objeto JSON vuelos
 - ▶ Métodos de array map- filter-reduce-every-some-find-copiar
 - ▶ Objetos creados por el usuario y objetos JSON
- ▼ Objetos-iterar y borrar prop-desestructurar-copiar-inmutables
 - ▶ borrar
 - ▼ Ejercicios
 - <> Ejercicio - copia objeto.html
 - <> Ejercicio - ejemplos iterar-desestructurar-borrar-copiar.html
- 101 011 Presentacion UT4.pdf

```
Ejercicio - copia array.html  Ejercicio - copia objeto.html  Ejercicio - ejemplos iterar-desestructurar-borrar-copiar.html  en arrays.html  + ▼
8      <input type="button" value="Crear objeto con copia=structuredClone(obj)" onclick="
      funcionObj2(persona)"><br>
10     <input type="button" value="Crear objeto con copia=JSON.parse(JSON.stringify(obj))"
      onclick="funcionObj3(persona)"><br>
11 </div id="demo"></div>
12 <script>
13   let persona={nombre:"Ana",provincia:"Almeria"}
14   function funcionObj1(obj){
15     let copia=obj;
16     copia.pais="España"
17     console.log("Original")
18     console.log(persona)
19     console.log("Copia")
20     console.log(copia)
21   }
22   function funcionObj2(obj){
23     let copia=structuredClone(obj);
24     copia.pais="España"
25     console.log("Original")
26     console.log(persona)
27     console.log("Copia")
28     console.log(copia)
29   }
30
31   function funcionObj3(obj){
32     let copia=JSON.parse(JSON.stringify(obj))
33     copia.pais="España"
34     console.log("Original")
35     console.log(persona)
36     console.log("Copia")
37     console.log(copia)
38   }
```

Line 32, Column 50

Tab Size: 4

HTML



Buscar

17:31 / 22:27



13°C



18:33

